

Simulación Secuencial y Paralela de un Ecosistema

Proyecto de Programación Paralela

Juan Diego Collazos Mejía
Juan Sebastián Garizao Puerto

23 de noviembre de 2025

Índice

1. Introducción	3
2. Descripción del Problema	3
2.1. Representación del ecosistema	4
2.2. Reglas de comportamiento de los conejos	4
2.3. Reglas de comportamiento de los zorros	4
2.4. Elementos estáticos: Rocas	5
2.5. Criterio determinista para seleccionar una celda	5
2.6. Resolución de conflictos	5
2.7. Formato de entrada y salida	6
3. Arquitectura General del Sistema	6
3.1. Componentes comunes	6
3.2. Arquitectura basada en listas de entidades	7
3.3. Arquitectura basada en filas y punteros	8
3.4. Doble buffer y flujo por generación	8
4. Implementaciones Secuenciales	9
4.1. Versión 1: Lista de Entidades	9
4.2. Versión 2: Decomposición por Filas	10
5. Implementaciones Paralelas	11
5.1. Versión 3: Paralelización basada en Listas de Entidades	11
5.2. Versión 4: Paralelización por Filas	12
5.3. Versión 5: Paralelización por Filas con Matriz Comprimida	13
6. Resultados y Análisis de Rendimiento	14
6.1. Metodología de Evaluación	14
6.2. Tiempos de ejecución	15
6.3. Comparación general	16
6.4. Speedup y Eficiencia	16
6.4.1. Lista de Entidades (Secuencial vs Paralela)	17

6.4.2.	Descomposición por Filas (Secuencial vs Paralela)	17
6.4.3.	Descomposición por Filas Optimizada (Secuencial vs Paralela Opt.)	17
6.5.	Análisis de los Resultados	17
6.5.1.	Comparación entre Versiones Secuenciales	18
6.5.2.	Comparación entre Versiones Paralelas	19
6.5.3.	Comparación Secuencial vs Paralela	20
7.	Discusión Técnica	21
7.1.	Reflexiones sobre las estructuras de datos	21
7.2.	Por qué unas estrategias fueron mejores que otras	21
7.3.	Limitaciones de cada arquitectura	22
7.4.	Decisiones sobre mutex y segmentación del trabajo	22
7.5.	Posibles mejoras futuras	23
8.	Conclusiones	23
8.1.	Conclusiones sobre las versiones secuenciales	23
8.2.	Conclusiones sobre las versiones paralelas	24
8.3.	Comparación secuencial vs paralelo	24
8.4.	Conclusiones finales del proyecto	25

1. Introducción

La simulación de ecosistemas es un problema clásico en el estudio de sistemas complejos, donde múltiples entidades interactúan siguiendo reglas locales que producen comportamientos globales no triviales. Estos modelos permiten observar dinámicas emergentes como ciclos depredador–presa, patrones espaciales, competencia por recursos y extinción de especies. Debido a la cantidad de interacciones por generación y a la naturaleza altamente paralelizable del problema, este tipo de simulaciones son una oportunidad ideal para aplicar técnicas de programación paralela.

El presente proyecto implementa una simulación basada en un ecosistema compuesto por tres tipos de entidades: **conejos**, **zorros** y **rocas**. Cada entidad sigue reglas estrictamente definidas sobre movimiento, reproducción, cacería y muerte por inanición, de acuerdo con el enunciado oficial del proyecto [?]. El entorno se modela como una matriz de $R \times C$ celdas, donde en cada generación los animales se mueven, interactúan y generan la siguiente configuración del ecosistema. Para evitar efectos de actualización simultánea se emplea un sistema de doble buffer en el que cada generación se construye a partir del estado anterior.

El objetivo principal del trabajo es comparar distintas estrategias de implementación tanto secuenciales como paralelas, analizando sus diferencias estructurales, sus ventajas, limitaciones y el impacto de cada diseño en el rendimiento. En particular, se desarrollaron cinco versiones del simulador:

- **Versión secuencial con lista de entidades:** enfoque basado en vectores de conejos y zorros para evitar recorrer completamente la matriz.
- **Versión secuencial por filas:** procesamiento directo de la matriz fila por fila.
- **Versión paralela con lista de entidades:** adaptación del primer enfoque utilizando hilos POSIX (pthread).
- **Versión paralela por filas:** reparto estático del ecosistema entre hilos, minimizando secciones críticas.
- **Versión paralela por filas optimizada:** versión mejorada que utiliza compresión de la matriz para procesar sólo celdas relevantes.

Estas implementaciones permiten comparar dos dimensiones: (1) la eficiencia algorítmica del enfoque secuencial y (2) la escalabilidad del paralelismo bajo distintos modelos de descomposición del problema. Además, se evalúa el rendimiento real mediante tiempos de ejecución, *speedup* y eficiencia, mostrando cómo las decisiones de diseño afectan el aprovechamiento de los recursos de hardware.

Finalmente, este informe presenta el análisis detallado de cada versión, explica las decisiones técnicas adoptadas y discute los resultados obtenidos, destacando las fortalezas y debilidades de cada aproximación. A partir de este estudio se obtiene una comprensión completa del problema tanto desde su dimensión algorítmica como desde el punto de vista de la programación paralela.

2. Descripción del Problema

El proyecto consiste en desarrollar un simulador que modele la evolución de un ecosistema poblado por tres tipos de entidades: **conejos**, **zorros** y **rocas**. El comportamiento

del sistema sigue las reglas definidas en el enunciado oficial del proyecto [?], donde cada entidad interactúa localmente en un entorno discreto representado por una matriz de dimensiones $R \times C$. En cada generación, los animales pueden moverse, reproducirse, cazar o morir, lo que produce nuevas configuraciones dinámicas del ecosistema.

2.1. Representación del ecosistema

El mundo se modela como una cuadrícula bidimensional donde cada celda puede contener exactamente una entidad o estar vacía. Las entidades posibles son:

- **RABBIT**: animal herbívoro que se mueve y reproduce rápidamente.
- **FOX**: depredador que caza conejos para sobrevivir.
- **ROCK**: elemento estático que no interactúa con otros objetos.

Cada entidad posee atributos locales como:

- *posición* (x, y) ,
- *edad de reproducción* (`local_proc`),
- *contador de hambre* en el caso de los zorros (`local_food`),
- *generación de nacimiento*.

La simulación inicia con un conjunto inicial de entidades y progresa durante un número fijo de generaciones N_GEN .

2.2. Reglas de comportamiento de los conejos

Los conejos siguen tres reglas fundamentales:

1. **Movimiento**: En cada generación intentan moverse a una celda adyacente libre (arriba, derecha, abajo o izquierda). Si existen múltiples opciones, la celda se selecciona con un criterio determinista basado en la generación G y la posición (x, y) .
2. **Reproducción**: Un conejo se reproduce cada `GEN_PROC_RABBITS` generaciones, siempre que se mueva. El conejo original y su cría reinician su contador de reproducción.
3. **Colisiones entre conejos**: Si dos o más conejos intentan ocupar la misma celda del buffer siguiente, sobrevive únicamente aquel con mayor antigüedad de reproducción.

2.3. Reglas de comportamiento de los zorros

Los zorros ejecutan un conjunto de reglas más complejo debido a la depredación:

1. **Cacería y movimiento**: Intentan primero moverse a una celda adyacente con un conejo. Si no hay conejos disponibles, intentan moverse a una celda vacía. Si ninguna opción es posible, permanecen en su posición.
2. **Muerte por inanición**: Si un zorro pasa `GEN_FOOD_FOXES` generaciones sin comer, muere antes de moverse.

3. **Reproducción:** Un zorro se reproduce cada `GEN_PROC_FOXES` generaciones, siempre que se mueva. La cría aparece en la celda previamente ocupada por el zorro original.
4. **Conflictos entre zorros:** Si varios zorros caen en la misma celda del siguiente buffer:
 - Sobrevive el de mayor antigüedad de reproducción.
 - En caso de empate, sobrevive el menos hambriento.

2.4. Elementos estáticos: Rocas

Las rocas funcionan como obstáculos permanentes:

- No se mueven.
- No se reproducen.
- Ninguna otra entidad puede ocupar su celda.

2.5. Criterio determinista para seleccionar una celda

Cuando un animal tiene P posibles celdas adyacentes válidas, se elige una de acuerdo con la fórmula:

$$\text{índice} = (G + x + y) \bmod P,$$

donde G es la generación actual y (x, y) es la posición de la entidad. Este criterio garantiza que, bajo las mismas condiciones iniciales, todas las versiones del simulador produzcan resultados idénticos.

2.6. Resolución de conflictos

El sistema se actualiza mediante un esquema de **doble buffer**. En cada generación:

1. Se procesan primero todos los conejos.
2. Luego se procesan todos los zorros.
3. Las rocas se copian directamente al siguiente buffer.

Los conflictos dentro del buffer siguiente incluyen:

- múltiples conejos queriendo la misma celda,
- múltiples zorros queriendo la misma celda,
- un zorro que entra en una celda ocupada por un conejo.

Estos se resuelven con reglas deterministas descritas anteriormente.

2.7. Formato de entrada y salida

La entrada del programa consiste en:

- Parámetros del sistema: `GEN_PROC_RABBITS`, `GEN_PROC_FOXES`, `GEN_FOOD_FOXES`, `N_GEN`, `R` y `C`.
- Número de entidades iniciales `N`.
- Lista de las entidades con su tipo y posición inicial.

La salida debe imprimir:

- Los mismos parámetros iniciales,
- seguido del estado final del ecosistema después de `N_GEN` generaciones.

Esta definición precisa permite reproducir el ecosistema de manera determinista y comparar resultados entre distintas versiones secuenciales y paralelas.

3. Arquitectura General del Sistema

La implementación del simulador se organiza alrededor de un conjunto de clases que encapsulan la lógica del ecosistema y los parámetros de simulación. Aunque existen dos estrategias principales de descomposición (por listas de entidades y por filas de la matriz), ambas comparten una base conceptual común: el uso de una matriz de celdas (**Mat**), una clase **Entity** para representar animales individuales y una clase **Param** que agrupa los parámetros globales de la simulación.

En esta sección se describe primero la arquitectura común y luego las diferencias estructurales entre la versión basada en listas de entidades y la versión basada en filas con punteros y matriz comprimida.

3.1. Componentes comunes

Ambas familias de implementaciones utilizan los siguientes tipos y clases básicas:

- **Tipo de entidad** (**EntityType**): Enumeración que clasifica el contenido de una celda como `EMPTY`, `ROCK`, `RABBIT` o `FOX`.
- **Entidad** (**Entity**): Representa un animal individual (conejo o zorro). En todas las versiones contiene:
 - posición (`x`, `y`),
 - contador de reproducción `local_proc`,
 - contador de hambre `local_food` (solo usado por zorros),
 - generación local o información de edad.
- **Parámetros globales** (**Param**): Clase inmutable que encapsula:
 - `GEN_PROC_RABBITS`,

- GEN_PROC_FOXES,
 - GEN_FOOD_FOXES,
 - N_GEN,
 - dimensiones de la matriz R y C.
- **Matriz del ecosistema (Mat):** Definida como un **vector** de **Row**, donde cada fila es a su vez un **vector** de **Cell**. El significado exacto de **Cell** depende de la estrategia de descomposición, como se explica a continuación.

Sobre estos componentes se construye la clase **Generation**, que representa el estado completo del ecosistema en una generación dada e incluye la lógica necesaria para imprimirlo y acceder a sus estructuras internas.

3.2. Arquitectura basada en listas de entidades

En la estrategia de descomposición por entidades, la clase **Cell** está definida como un contenedor ligero que almacena:

- el tipo de la entidad presente en la celda (**EntityType**),
- un índice entero **index** que apunta a la posición de dicha entidad en su vector correspondiente.

De esta forma, la matriz **Mat** no contiene las entidades en sí, sino solo referencias indirectas hacia los vectores:

- `vector<Entity>rabbits;`
- `vector<Entity>foxes;`
- `vector<Entity>rocks;` (constante).

La clase **Generation** en esta arquitectura mantiene:

- la matriz **ecosystem** de **Cell**,
- los vectores de conejos y zorros, que se utilizan para iterar rápidamente sobre todas las entidades activas sin recorrer la matriz completa,
- un contador de generación local.

Este diseño favorece operaciones donde interesa procesar directamente el conjunto de animales (conejos y zorros) y usar la matriz únicamente como estructura auxiliar para conocer ocupación y resolver conflictos. Desde el punto de vista algorítmico, la ventaja principal es que la complejidad de actualizar entidades depende del número de animales y no del tamaño total de la malla.

En las versiones secuenciales, esta arquitectura resulta especialmente eficiente y relativamente sencilla de mantener: cada paso de la simulación recorre las listas de conejos o zorros, actualiza sus posiciones y luego reconstruye la matriz de acuerdo con los índices actualizados.

3.3. Arquitectura basada en filas y punteros

En la estrategia de descomposición por filas, la definición de `Cell` cambia de forma significativa. En lugar de almacenar un índice, la celda contiene:

- el tipo de la entidad (`EntityType`),
- un puntero `Entity*` hacia la instancia concreta almacenada en la propia matriz.

Así, la matriz `Mat` pasa a ser la estructura principal donde residen las entidades, y las celdas almacenan punteros a objetos dinámicamente reservados. En esta arquitectura:

- no existe dentro de `Generation` un `vector` explícito de conejos o zorros,
- el recorrido del sistema se realiza fila por fila, celda por celda, aplicando las reglas de movimiento y reproducción según el tipo de entidad encontrada.

La clase `Generation` para esta familia incluye:

- la matriz `ecosystem` de `Cell` con punteros,
- un contador de generación local,
- y, en la versión optimizada, una estructura adicional `vector<vector<int>>` `eco_compress`.

La estructura `eco_compress` actúa como una *matriz comprimida*: para cada fila almacena un vector con las columnas donde existe alguna entidad viva. De este modo, la versión optimizada por filas no recorre todas las celdas, sino únicamente aquellas posiciones activas, reduciendo el coste de procesamiento cuando el ecosistema es disperso.

3.4. Doble buffer y flujo por generación

Independientemente de la estrategia de descomposición, todas las versiones comparten la misma idea de **doble buffer**:

- Se mantienen dos objetos `Generation`: uno *actual* (`current`) y uno *siguiente* (`next`).
- En cada generación, se lee únicamente del buffer actual y se escriben los resultados en el buffer siguiente.
- Al finalizar el procesamiento de conejos y zorros, se intercambian los punteros (`swap(current, next)`) sin necesidad de copiar toda la estructura.

El flujo básico por generación es:

1. Mover y procesar conejos, escribiendo en el buffer siguiente.
2. Intercambiar buffers.
3. Mover y procesar zorros, escribiendo en el nuevo buffer siguiente.
4. Intercambiar buffers nuevamente.
5. Actualizar el contador de generación.

En la arquitectura por listas, este proceso se apoya en los vectores de entidades para iterar; en la arquitectura por filas, el procesamiento se hace recorriendo directamente las filas de la matriz (y, en la versión optimizada, aprovechando la estructura `eco_compress` para saltarse celdas vacías).

Esta organización permite separar claramente la lógica del ecosistema de las decisiones sobre descomposición y paralelización, y garantiza que, sin importar la versión, el sistema preserve la consistencia por generación.

4. Implementaciones Secuenciales

El simulador cuenta con dos versiones secuenciales que representan dos estrategias distintas de recorrer y actualizar el ecosistema. Ambas producen resultados funcionalmente idénticos, pero difieren de manera importante en su estructura interna, complejidad, y sobre todo en cómo se comportan al intentar paralelizarlas posteriormente.

Las dos aproximaciones son:

- **Versión secuencial basada en listas de entidades**
- **Versión secuencial basada en descomposición por filas**

En esta sección se describe cada una en detalle y se analizan sus ventajas y limitaciones intrínsecas.

4.1. Versión 1: Lista de Entidades

Esta primera implementación secuencial se basa en mantener listas explícitas de conejos, zorros y rocas dentro de la clase `Generation`. La matriz del ecosistema (`Mat`) no almacena punteros a entidades, sino únicamente un par:

- el tipo de la entidad (`EntityType`),
- un índice entero que apunta a la posición de esa entidad dentro del vector correspondiente.

Este diseño permite procesar únicamente las entidades activas sin recorrer la matriz completa, lo cual es ventajoso cuando la densidad del ecosistema es baja o cuando el número de animales es muy inferior al tamaño del mundo.

Flujo de ejecución

El flujo de una generación en esta versión es:

1. Iterar sobre el vector de conejos: aplicar movimiento, reproducción, resolver conflictos, y escribir cada resultado en el buffer siguiente.
2. Intercambiar los buffers.
3. Iterar sobre el vector de zorros: aplicar movimiento, cacería, reproducción, resolver conflictos y escribir resultados en el siguiente buffer.
4. Intercambiar buffers nuevamente.

5. Actualizar la generación local.

En todos los casos, la matriz **ecosystem** se actualiza indirectamente a partir de los vectores, asignando a cada celda un **Cell** que contiene solamente tipo e índice.

Ventajas

- Excelente rendimiento secuencial: solo se procesan entidades vivas, no toda la matriz.
- Estructura de datos limpia para manipular entidades.
- Simplicidad para reconstruir el estado del ecosistema tras cada paso.

Limitaciones

- No es directa de paralelizar: múltiples hilos insertando en vectores compartidos generan condiciones de carrera y bloqueos.
- El índice almacenado en la celda requiere mantener coherencia entre posiciones del vector y la matriz.
- La resolución de conflictos entre hilos implicaría usar mutex globales costosos.

A pesar de estas limitaciones para el paralelismo, esta versión es la base más clara y eficiente para la versión secuencial del simulador.

4.2. Versión 2: Decomposición por Filas

La segunda versión secuencial abandona por completo los vectores de conejos y zorros. En su lugar, toda la información de las entidades se almacena directamente dentro de la matriz **Mat**, donde cada **Cell** contiene:

- el tipo de la entidad,
- un puntero **Entity*** a la instancia concreta.

Esto transforma la matriz en la estructura de datos principal del sistema, mientras que las entidades existen únicamente dentro de las celdas del ecosistema.

Flujo de ejecución

El procesamiento secuencial en esta versión sigue el flujo:

1. Recorrer fila por fila y celda por celda.
2. Cuando se encuentra un conejo, aplicar movimiento y reproducción; insertar el resultado directamente en el buffer siguiente.
3. Intercambiar buffers.
4. Recorrer nuevamente toda la matriz, esta vez procesando zorros.
5. Intercambiar buffers.

En este diseño la matriz es recorrida completamente, incluso si solo una fracción de las celdas contiene entidades.

Ventajas

- Mucho más fácil de paralelizar por filas: cada hilo procesa un bloque de filas sin necesidad de acceder a vectores compartidos.
- Evita la complejidad de mantener una relación índice-celda.
- La matriz se interpreta de manera directa, sin estructuras auxiliares.

Limitaciones

- Menor eficiencia secuencial comparada con la versión basada en listas, especialmente si el ecosistema es disperso.
- El uso de memoria dinámica y punteros requiere limpieza manual en cada generación.
- Implica recorrer toda la matriz aun cuando muchas celdas estén vacías.

A pesar de estas limitaciones, esta versión se convierte en la base ideal para las versiones paralelas del proyecto, pues su estructura por filas permite dividir el trabajo de forma natural y con poca interacción entre hilos.

5. Implementaciones Paralelas

A partir de las dos arquitecturas secuenciales descritas anteriormente, se desarrollaron tres versiones paralelas con el objetivo de analizar diferentes estrategias de descomposición de datos y observar su impacto en el rendimiento. Cada una de estas versiones parte de un enfoque distinto y presenta ventajas y limitaciones específicas, especialmente en términos de sincronización, escrituras concurrentes y reparto de trabajo entre hilos.

Las tres variantes paralelas implementadas fueron:

- **Versión paralela basada en listas de entidades.**
- **Versión paralela basada en descomposición por filas.**
- **Versión paralela por filas optimizada con matriz comprimida.**

En esta sección se describe en detalle cada una de ellas.

5.1. Versión 3: Paralelización basada en Listas de Entidades

Esta versión intenta paralelizar directamente la arquitectura secuencial basada en vectores de conejos y zorros. El mundo sigue siendo una matriz **Mat**, cuyas celdas almacenan tipo e índice, pero la iteración se divide entre hilos asignando a cada uno un bloque del vector de entidades.

Estrategia de paralelización

Para cada generación:

1. Se divide el vector de conejos entre los hilos. Cada hilo procesa un subrango del vector.
2. Una vez terminan todos los hilos, se intercambia el buffer.
3. Se repite el proceso para los zorros.

Cada hilo ejecuta sobre su propio conjunto de entidades todas las reglas: movimiento, reproducción, cacería y escritura en el buffer siguiente.

Problemas fundamentales

Aunque este enfoque parece natural, en la práctica presenta problemas severos de sincronización:

- **Vectores de escritura compartida.** Los hilos necesitan insertar nuevas entidades (crías) en los vectores de conejos o zorros, lo que requiere proteger cada inserción con un mutex.
- **Conflictos en la matriz.** Múltiples hilos pueden intentar escribir en la misma celda del buffer siguiente, generando colisiones que requieren mutex adicionales.
- **Coherencia índice-celda.** Debido a que las celdas almacenan índices, cualquier reordenamiento, inserción o borrado puede dejar celdas apuntando a posiciones incorrectas.
- **Falsos conflictos.** Los hilos se bloquean entre sí incluso si están trabajando en regiones lógicamente independientes. El costo de los mutex domina el tiempo de ejecución.

En consecuencia, aunque la versión es funcional, no escala adecuadamente. Los costos de sincronización representan un cuello de botella inevitable.

5.2. Versión 4: Paralelización por Filas

Esta versión parte de la arquitectura secuencial basada en una matriz cuyos `Cell` contienen punteros directos a las entidades. Esta estructura permite dividir la matriz horizontalmente: cada hilo recibe un conjunto fijo de filas sobre las cuales trabajar sin necesidad de acceder a vectores compartidos.

Estrategia de paralelización

- La matriz se divide en T bloques de filas aproximadamente iguales, donde T es el número de hilos.
- Cada hilo procesa únicamente sus filas:
 - movimiento de conejos en sus filas,

- movimiento de zorros en sus filas,
- escritura en el buffer siguiente.
- Solo en las fronteras entre bloques puede ser necesaria comunicación mínima: comprobar si un animal cruza a la fila de otro hilo.

Ventajas

- **Escalabilidad natural:** Cada hilo trabaja en un segmento continuo de memoria, mejorando la localidad de caché.
- **Sin vectores compartidos:** No hay inserciones en contenedores globales.
- **Muy poca sincronización:** Solo es necesario proteger conflictos en celdas adyacentes entre hilos.

Limitaciones

- Cada hilo debe recorrer todas sus filas, incluso si están vacías.
- La sobrecarga por movimiento entre zonas de trabajo puede generar revisiones adicionales en las fronteras.

En comparación con la versión basada en listas, esta versión mejora significativamente la escalabilidad y reduce la necesidad de sincronización.

5.3. Versión 5: Paralelización por Filas con Matriz Comprimida

La versión final introduce una optimización adicional usando la estructura `eco_compress`, un *índice de actividad por fila*. Para cada fila se almacena únicamente un vector de las columnas donde existe al menos una entidad viva.

En lugar de recorrer todas las celdas de la matriz, el algoritmo paralelizado recorre:

solo las posiciones activas registradas en *eco_compress*.

Estrategia de ejecución

- Cada hilo sigue procesando un bloque de filas, pero ahora:
 - Solo recorre los índices activos de su bloque.
 - Actualiza la matriz comprimida del buffer siguiente.
- Las celdas vacías se ignoran por completo, reduciendo el número de operaciones necesarias.

Ventajas

- **Reducción drástica del número de accesos:** Ideal cuando el ecosistema es disperso o cuando hay muchas rocas.
- **Localidad superior:** Cada hilo opera sobre un número menor de posiciones contiguas.
- **Menor carga de trabajo y mayor paralelismo efectivo.**

Limitaciones

- La estructura comprimida debe regenerarse en cada generación.
- Requiere coordinación adicional para mantener coherencia entre `ecosystem` y `eco_compress`.

6. Resultados y Análisis de Rendimiento

En esta sección se evalúan las cinco versiones del simulador bajo múltiples configuraciones del ecosistema. Los experimentos se realizaron utilizando los mismos archivos de entrada para todas las implementaciones con el fin de garantizar comparabilidad directa. Las métricas consideradas incluyen:

- **Tiempo Wall** (tiempo real transcurrido),
- **Tiempo CPU** (suma del tiempo empleado por todos los hilos),
- **Speedup**,
- **Eficiencia**.

Los resultados permiten observar el impacto del tamaño del ecosistema, del número de generaciones simuladas y de la arquitectura de cada implementación.

6.1. Metodología de Evaluación

Para cada una de las cinco versiones se ejecutaron los siguientes archivos de prueba:

Cuadro 1: Archivos de prueba utilizados en la evaluación del simulador

ID	Archivo	Tamaño	N. generaciones
A1	5x5.txt	5×5	6
A2	10x10.txt	10×10	100
A3	20x20.txt	20×20	1000
A4	100x100.txt	100×100	10000
A5	100x100_unbal01.txt	100×100	10000
A6	100x100_unbal02.txt	100×100	10000
A7	200x200.txt	200×200	10000

Los casos *unbalanced* representan ecosistemas con distribuciones no uniformes de entidades, lo cual afecta la cantidad de celdas activas y, por tanto, la carga de trabajo real.

Todas las pruebas se ejecutaron en el mismo entorno físico, con las mismas condiciones iniciales y sin modificaciones en los parámetros del sistema.

6.2. Tiempos de ejecución

A continuación se presentan tablas que resumen los tiempos Wall de las cinco versiones.

Cuadro 2: Tiempos Wall (s) — Versión Secuencial Lista de Entidades

Entrada	Wall (s)
A1	6.12e-05
A2	0.00226
A3	0.09884
A4	34.16
A5	24.47
A6	33.78
A7	148.90

Cuadro 3: Tiempos Wall (s) — Versión Paralela Lista de Entidades

Entrada	Wall (s)
A1	0.00404
A2	0.2829
A3	2.8437
A4	59.66
A5	47.02
A6	58.50
A7	170.28

Cuadro 4: Tiempos Wall (s) — Versión Secuencial Filas

Entrada	Wall (s)
A1	9.69e-05
A2	0.00480
A3	0.3273
A4	48.69
A5	46.04
A6	45.56
A7	225.47

Cuadro 5: Tiempos Wall (s) — Versión Paralela Filas

Entrada	Wall (s)
A1	0.00437
A2	0.12076
A3	1.4885
A4	26.84
A5	24.55
A6	27.48
A7	77.72

Cuadro 6: Tiempos Wall (s) — Versión Paralela Filas Optimizada

Entrada	Wall (s)
A1	0.00454
A2	0.22926
A3	1.3673
A4	27.93
A5	21.11
A6	27.25
A7	81.68

6.3. Comparación general

Los resultados permiten observar varios patrones globales:

- Para tamaños pequeños (5x5, 10x10), **todas las versiones paralelas son más lentas** debido al overhead de creación y sincronización de hilos.
- Para tamaños medianos (20x20, 100x100), las versiones paralelas por filas comienzan a superar a sus versiones secuenciales.
- Para tamaños grandes (200x200), la mejor versión es la paralela por filas, seguida de la paralela por filas optimizada.
- La versión paralela basada en listas de entidades nunca supera a la secuencial equivalente, debido a su altísimo costo de sincronización.

6.4. Speedup y Eficiencia

El desempeño de las versiones paralelas se evaluó mediante dos métricas:

$$S = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}}}, \quad E = \frac{S}{P}$$

donde $P = 8$ corresponde al número de hilos utilizados. El **speedup** indica la aceleración alcanzada, mientras que la **eficiencia** mide qué tan bien se aprovechan los hilos disponibles.

A continuación se presentan las tablas comparativas entre cada versión paralela y su versión secuencial correspondiente.

6.4.1. Lista de Entidades (Secuencial vs Paralela)

Cuadro 7: Speedup y Eficiencia — Lista de Entidades

Entrada	Seq (s)	Par (s)	Speedup	Eficiencia
A1	6.12e-05	0.00404	0.015	0.0019
A2	0.002264	0.282905	0.008	0.0010
A3	0.09884	2.84369	0.034	0.0043
A4	34.1607	59.6606	0.57	0.071
A5	24.4722	47.0224	0.52	0.065
A6	33.7805	58.5040	0.57	0.071
A7	148.903	170.289	0.87	0.109

6.4.2. Descomposición por Filas (Secuencial vs Paralela)

Cuadro 8: Speedup y Eficiencia — Descomposición por Filas

Entrada	Seq (s)	Par (s)	Speedup	Eficiencia
A1	9.69e-05	0.004374	0.022	0.0027
A2	0.004799	0.120762	0.039	0.0049
A3	0.327377	1.48852	0.22	0.027
A4	48.698	26.8427	1.81	0.226
A5	46.0499	24.5499	1.87	0.234
A6	45.5658	27.4811	1.65	0.206
A7	225.479	77.7293	2.90	0.363

6.4.3. Descomposición por Filas Optimizada (Secuencial vs Paralela Opt.)

Cuadro 9: Speedup y Eficiencia — Descomposición por Filas Optimizada

Entrada	Seq (s)	Par-Opt (s)	Speedup	Eficiencia
A1	9.69e-05	0.004540	0.021	0.0026
A2	0.004799	0.229260	0.021	0.0026
A3	0.327377	1.36732	0.23	0.029
A4	48.698	27.9346	1.74	0.218
A5	46.0499	21.1184	2.18	0.272
A6	45.5658	27.2568	1.67	0.208
A7	225.479	81.6864	2.76	0.345

6.5. Análisis de los Resultados

El análisis cuantitativo obtenido a partir de los tiempos de ejecución, el speedup y la eficiencia permite comparar con claridad el comportamiento de las cinco versiones del simulador. A continuación se discuten por separado las implementaciones secuenciales, las paralelas, y finalmente la relación entre cada versión secuencial y su correspondiente versión paralela.

6.5.1. Comparación entre Versiones Secuenciales

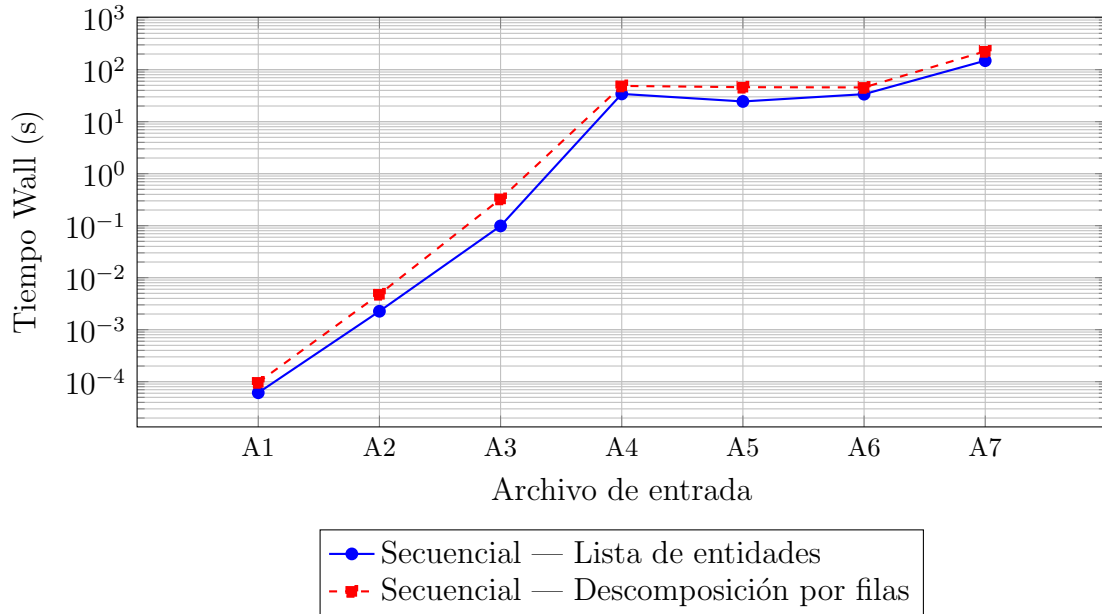


Figura 1: Tiempos Wall de las versiones secuenciales según el archivo de entrada.

Al comparar las dos implementaciones secuenciales —lista de entidades y descomposición por filas— se observa un patrón consistente:

- Para tamaños pequeños (hasta 20x20), ambas versiones muestran tiempos muy reducidos, con diferencias marginales debido a que el costo del recorrido de la matriz es mínimo.
- Para tamaños medianos y grandes (100x100 y 200x200), la versión basada en **lista de entidades es más rápida** que la descomposición por filas. Esto se debe a que sólo recorre las entidades vivas, mientras que la versión por filas debe recorrer toda la matriz, incluso celdas vacías.
- En casos *unbalanced*, donde hay menos entidades activas, la diferencia a favor de la lista de entidades aumenta, ya que la matriz por filas sigue recorriendo zonas enteras sin actividad.

En general, la estrategia secuencial más eficiente es la versión basada en listas de entidades, debido a su complejidad proporcional al número de animales en lugar del tamaño total del ecosistema.

6.5.2. Comparación entre Versiones Paralelas

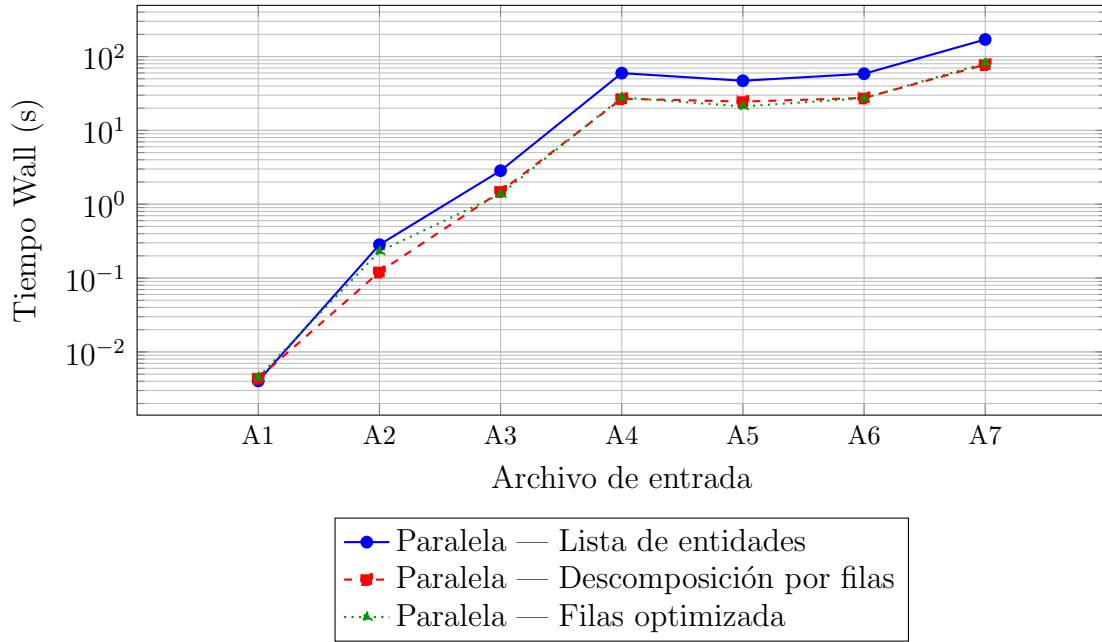


Figura 2: Tiempos Wall de las versiones paralelas según el archivo de entrada.

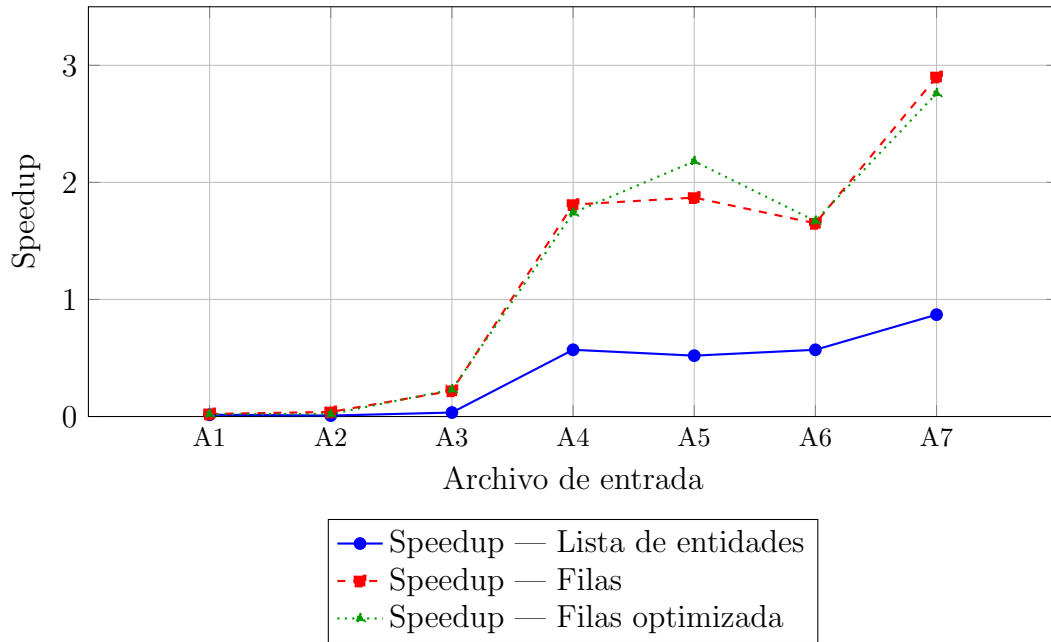


Figura 3: Speedup obtenido por cada arquitectura paralela con respecto a su versión secuencial.

Las tres versiones paralelas muestran diferencias claras en comportamiento, principalmente influenciadas por la estructura de datos utilizada y la cantidad de sincronización requerida.

Paralelización basada en lista de entidades. Esta versión presenta los peores resultados entre todas las paralelas:

- En todos los casos, el speedup es menor que 1.
- La eficiencia nunca supera el 10 %.
- La alta contención provocada por inserciones concurrentes en vectores, actualizaciones de índices y múltiples mutex hace que el paralelismo sea contraproducente.

Paralelización por filas. En contraste, la descomposición por filas logra speedups positivos y consistentes:

- Para 100x100 se obtienen speedups entre 1.6 y 1.9.
- Para 200x200 se alcanza el mejor resultado del proyecto:

$$S_{200 \times 200} = 2,90, \quad E = 0,363.$$

- La eficiencia mejora con el tamaño del ecosistema, ya que el costo de creación y sincronización de hilos se amortiza más fácilmente.

Paralelización por filas optimizada. Esta versión incorpora la matriz comprimida, lo cual genera beneficios cuando el ecosistema tiene pocas celdas activas:

- En matrices *unbalanced*, la optimización produce speedup mayor que la versión no optimizada (por ejemplo, 2.18 frente a 1.87 en el caso 100x100_unbal01).
- En matrices densas, la optimización no mejora de forma significativa, pues casi todas las celdas están activas y `eco_compress` almacena toda la fila de igual manera.

6.5.3. Comparación Secuencial vs Paralela

Al comparar cada arquitectura con su contraparte paralela, se puede analizar lo siguiente:

Lista de entidades.

- La versión paralela es consistentemente más lenta.
- La razón es estructural: la cantidad de comunicación entre hilos y la necesidad de proteger accesos a vectores hace imposible obtener speedup.
- Incluso para 200x200, el speedup de 0.87 indica pérdida de rendimiento.

Descomposición por filas.

- La versión paralela mejora la secuencial de manera clara en tamaños medianos y grandes.
- La eficiencia de 20 %-36 % es razonable para un escenario con partes que no pueden paralelizarse (según la ley de Amdahl).

Descomposición por filas optimizada.

- Aunque la versión optimizada no siempre supera a la paralela normal, sí lo hace cuando el ecosistema es disperso o altamente irregular.
- En el caso 100x100_unbal01, la eficiencia aumenta de 23.4 % a 27.2 %.

7. Discusión Técnica

El desempeño observado en las distintas versiones del simulador permite analizar la calidad de las decisiones de diseño adoptadas, así como identificar los factores que influyeron positiva o negativamente en el rendimiento. En esta sección se discuten los aspectos más relevantes desde una perspectiva técnica: estructura de datos, granularidad del paralelismo, contención, localidad, sincronización y oportunidades de optimización futuras.

7.1. Reflexiones sobre las estructuras de datos

La diferencia fundamental entre las arquitecturas empleadas radica en la forma de representar el ecosistema:

- **Lista de entidades:** estructura eficiente en secuencial, pues la complejidad del procesamiento depende del número de animales vivos, no del tamaño completo de la matriz.
- **Descomposición por filas con punteros:** estructura óptima para paralelismo, donde las celdas contienen referencias directas a objetos **Entity** y se evita mantener listas globales.

El cambio entre indexado por lista y punteros fue determinante. En la versión secuencial, usar índices en la matriz permite recorrer entidades de forma compacta y eficiente. Sin embargo, en paralelo, este diseño introduce conflictos entre hilos que compiten por modificar vectores compartidos.

Por el contrario, en la descomposición por filas, los punteros permiten que cada hilo trabaje sobre su propia región sin afectar a las estructuras de los demás, minimizando así el riesgo de condiciones de carrera y el uso de mutexes.

7.2. Por qué unas estrategias fueron mejores que otras

Las métricas experimentales confirman lo siguiente:

- La **lista de entidades** es superior en secuencial porque procesa únicamente entidades activas. Sin embargo, su paralelización resulta ineficaz debido a la contención generada en las operaciones sobre vectores.
- La **descomposición por filas** es la única arquitectura que escala de manera consistente en paralelo, gracias a que divide el trabajo de forma natural y con mínima interacción entre hilos.

- La versión **optimizada** con matriz comprimida beneficia ecosistemas dispersos, donde la cantidad de entidades activas es pequeña en comparación con el área total. En esos casos, reduce significativamente el número de celdas que los hilos deben visitar.

En síntesis, el rendimiento está directamente relacionado con el balance entre: (i) cantidad de trabajo independiente para cada hilo, (ii) necesidad de acceder a estructuras globales, (iii) costo de sincronización necesaria.

7.3. Limitaciones de cada arquitectura

Cada enfoque presenta limitaciones inherentes:

Lista de entidades.

- El acceso concurrente a vectores globales introduce necesidades constantes de sincronización.
- Resolver conflictos requiere mutexes amplios que bloquean regiones grandes del código.
- La coherencia entre índices en celdas y posiciones en vectores se vuelve compleja cuando varios hilos modifican los vectores.

Descomposición por filas.

- Recorre toda la matriz incluso si está mayormente vacía.
- El borde entre bloques de filas requiere inspecciones adicionales para manejar correctamente entidades que cruzan zonas.

Optimización con matriz comprimida.

- La reconstrucción de `eco_compress` en cada generación tiene un costo adicional.
- Cuando la matriz es densa, `eco_compress` aporta poco o ningún beneficio.

7.4. Decisiones sobre mutex y segmentación del trabajo

La estrategia de paralelización influye decisivamente en cómo y cuándo se deben utilizar mutexes:

- En la arquitectura basada en listas, es necesario proteger:
 - inserciones en vectores de conejos y zorros,
 - actualizaciones de índices en la matriz,
 - resolución de conflictos entre entidades.

Esto genera una alta contención, reduce la paralelización efectiva y explica por qué los speedups son menores que 1 en todos los casos.

- En la descomposición por filas, la segmentación natural permite que cada hilo trabaje en una región del ecosistema completamente independiente. Solo se requieren mutexes mínimos en los bordes entre bloques, lo cual reduce drásticamente el costo de sincronización.
- La versión optimizada mantiene la misma segmentación, pero reduce el tamaño del espacio que cada hilo debe recorrer, lo que mejora la distribución del trabajo y disminuye aún más la posible contención.

7.5. Posibles mejoras futuras

Existen varias direcciones en las que este proyecto puede extenderse:

- **Paralelización dinámica:** En lugar de dividir filas de manera estática, usar colas de tareas puede evitar desbalance cuando algunas regiones del ecosistema tienen más actividad que otras.
- **Reducción de contención:** Para la versión de listas, se podrían emplear estructuras lock-free o buffers por hilo seguidos de una fusión final controlada.
- **Uso de afinidad de CPU y NUMA-awareness:** Asignar subconjuntos de filas a hilos fijos mejora la localidad de caché.
- **Vectorización:** Algunas operaciones repetitivas sobre la matriz podrían beneficiarse de instrucciones SIMD.
- **GPU paralelismo:** La arquitectura por filas es ideal para ser paralelizada en una GPU, donde cada celda o grupo de celdas puede procesarse con miles de threads.

8. Conclusiones

El desarrollo de las cinco versiones del simulador del ecosistema permitió comparar de forma integral cómo las decisiones de diseño, la estructura de datos y la estrategia de paralelización influyen en el rendimiento de un sistema altamente iterativo y dependiente del acceso a memoria. A partir del análisis de tiempos, speedup y eficiencia, es posible extraer las conclusiones principales del proyecto.

8.1. Conclusiones sobre las versiones secuenciales

Entre las dos implementaciones secuenciales, la arquitectura basada en **listas de entidades** resultó ser la más eficiente en prácticamente todos los escenarios. Esto se debe principalmente a que:

- el costo computacional depende del número de entidades vivas y no del tamaño total del ecosistema, y
- evita recorrer celdas vacías, lo que reduce trabajo en casos densos y especialmente en los *unbalanced*.

La versión secuencial basada en descomposición por filas realizó recorridos completos de la matriz en cada generación, lo cual incrementó los tiempos para ecosistemas de tamaño medio o grande. No obstante, su estructura es más simple y demuestra ser la base ideal para la paralelización.

8.2. Conclusiones sobre las versiones paralelas

El paralelismo evidenció contrastes muy marcados entre las arquitecturas:

Paralelización basada en listas de entidades. Esta versión nunca superó a su contraparte secuencial. El speedup se mantuvo siempre por debajo de 1 y la eficiencia por debajo del 10 %. La contención generada por vectores compartidos, actualizaciones de índices y múltiples mutexes volvió inefectivo el paralelismo.

Paralelización por filas. La descomposición por filas fue la estrategia más exitosa. Dividir el ecosistema en bloques de filas permitió:

- minimizar la interacción entre hilos,
- mejorar significativamente la localidad de memoria,
- reducir el uso de sincronización.

Para ecosistemas grandes (200x200), esta versión alcanzó el mejor rendimiento de todo el proyecto, con:

$$S_{\text{máx}} = 2,90, \quad E_{\text{máx}} = 0,363.$$

Valores consistentes con la fracción paralelizable del problema (ley de Amdahl).

Paralelización por filas optimizada. La versión optimizada con `eco_compress` mostró beneficios claros cuando el ecosistema era disperso o no uniforme. En particular:

$$S = 2,18 \quad \text{para el caso } 100 \times 100_{\text{unbal01}},$$

superando a la versión paralela por filas estándar. Sin embargo, en ecosistemas densos la mejora fue limitada, ya que la matriz comprimida contenía prácticamente la misma información que la matriz original.

8.3. Comparación secuencial vs paralelo

Las conclusiones generales de esta comparación son:

- La arquitectura secuencial más rápida no es necesariamente la que se paraleliza mejor. La lista de entidades es óptima en secuencial pero ineficiente en paralelo.
- La arquitectura por filas, aunque más lenta en secuencial, fue la única que permitió obtener speedups significativos.
- La paralelización solo ofrece ventajas reales a partir de tamaños medianos; en problemas pequeños, el overhead supera las ganancias.

La eficiencia alcanzada (20–36 %) es razonable para un problema con partes intrínsecamente secuenciales, altas dependencias de datos por generación y accesos frecuentes a memoria.

8.4. Conclusiones finales del proyecto

A partir de los resultados y el análisis técnico, se pueden establecer las siguientes conclusiones globales:

- La **descomposición por filas** es la estrategia más adecuada para este tipo de simulaciones, equilibrando simplicidad, escalabilidad y un uso eficiente del paralelismo del hardware.
- La **lista de entidades** es excelente en secuencial pero presenta una arquitectura incompatible con paralelismo eficiente en memoria compartida.
- El uso de estructuras auxiliares como **eco_compress** abre la puerta a mejoras basadas en sparsity, mostrando que adaptar la estructura de datos al comportamiento del ecosistema produce beneficios reales.
- El proyecto demuestra que la estructura de datos es tan importante como el algoritmo o el modelo de paralelización. Cambiar la representación interna del ecosistema fue la clave para lograr escalabilidad.
- La implementación paralela final ofrece mejoras sustanciales, pero aún puede refinarse mediante paralelismo dinámico, técnicas lock-free, vectorización o incluso versiones híbridas CPU/GPU.

En conjunto, el proyecto evidencia la importancia de analizar la interacción entre estructura de datos, acceso a memoria y paralelización. Las decisiones tomadas permitieron comprender por qué algunos diseños escalan y otros no, y muestran el valor de adaptar una arquitectura a las características del hardware y del problema subyacente.